# How to Implement Atomicity and Durability in Transactions

## 1. Overview

Two important ACID properties are:

- **Atomicity** → Transaction happens fully or not at all
- **Durability** → Once committed, data remains permanent

These are mainly handled by the **Recovery Mechanism** of a DBMS.

```None
Recovery Manager

→ Handles crash recovery

→ Undo incomplete work

→ Redo committed work
```

## 2. Main Techniques Used

Databases commonly use:

| Method | Purpose |
|---|---|
| Shadow Copying | Older simple technique |

| | |
|---|---|
| Log-Based Recovery | Modern practical technique |
| Checkpointing | Faster recovery |
| Replication | Extra durability |

# 3. Shadow Copy Scheme

# Concept

Create a full copy of the database before changes.

```None
Old DB = original copy

New DB = modified copy

db-pointer = points to active DB
```

Only after successful completion does pointer switch to new copy.

# 4. Steps of Shadow Copying

## Step 1

Transaction T starts.

## Step 2

Create full copy of database.

## Step 3

Apply all updates to new copy only.

Original DB remains unchanged.

## Step 4: If Abort Happens

Delete new copy.

Old DB still valid.

## Step 5: If Commit Happens

```None
1. Write new copy fully to disk

2. Update db-pointer

3. New copy becomes official DB

4. Delete old copy later
```

Commit considered complete when pointer update is safely stored.

# 5. Atomicity in Shadow Copy

If failure occurs before pointer update:

None
```
db-pointer still points to old DB
```

So partial changes disappear.

None
```
0% visible
```

If pointer updated:

None
```
100% visible
```

Hence atomicity achieved.

# 6. Durability in Shadow Copy

If commit pointer written successfully:

After restart:

None
```
db-pointer → new DB copy
```

Committed data survives crash.

That gives durability.

# 7. Problem with Shadow Copy

Copies entire DB every transaction
Very slow for large databases
High disk usage
Poor concurrency

So modern DBs rarely use full shadow copying.

# 8. Log-Based Recovery (Modern Method)

Most real systems use transaction logs.

Examples:

- PostgreSQL (WAL)
- MySQL (redo/undo logs)
- Oracle Database

# 9. What Is a Log?

A log stores records like:

```None
Transaction ID

Operation

Old value

New value

Timestamp

Commit marker
```

Stored in stable storage.

# 10. Key Rule: Write-Ahead Logging (WAL)

```None

Write log first

Then write database page
```

Very important principle.

If crash happens, recovery uses logs.

# 11. Deferred Modification

# Meaning

Database changes delayed until transaction commits.

Steps:

```None
1. Record intended updates in log

2. Do not update DB immediately

3. On commit → apply writes
```

## Failure Cases

Before commit:

```None
Ignore logs
```

After commit but partial write:

```None
REDO committed transaction
```

# 12. Immediate Modification

# Meaning

Database may be updated before commit.

Uncommitted changes may exist temporarily.

Logs store:

```None
Old value

New value
```

## Failure Cases

Before commit:

```None
UNDO using old value
```

After commit:

```None
REDO using new value
```

# 13. How Atomicity Works with Logs

If transaction fails midway:

```None
UNDO all partial changes
```

So either all changes happen or none.

# 14. How Durability Works with Logs

If commit record safely stored:

Even if crash happens immediately:

```None
REDO committed changes during recovery
```

Committed transaction persists.

# 15. Example: Bank Transfer

```None
A = 500

B = 300

Transfer 50
```

Log:

```None
T1: A old=500 new=450

T1: B old=300 new=350

T1: COMMIT
```

Crash after commit?

Recovery replays updates.

# 16. Comparison

| Method | Atomicity | Durability | Performance |
|---|---|---|---|
| Shadow Copy | Yes | Yes | Poor |
| Log Recovery | Yes | Yes | Excellent |
| WAL + Checkpoint | Yes | Yes | Best practical |

# 17. Checkpointing (Extra Important)

Periodically save clean state:

```None
checkpoint created
```

Recovery starts from checkpoint + logs after it.

Faster restart.

# 18. Interview Answer

**Q: How are atomicity and durability implemented?**

> Atomicity is achieved through undo mechanisms that roll back incomplete transactions. Durability is achieved through write-ahead logs, checkpoints, and persistent storage so committed transactions survive crashes.

# 19. Short Revision Notes

```None
Atomicity:
UNDO incomplete txns


Durability:
REDO committed txns


Techniques:
- Shadow copy (old)
- WAL logs (modern)
- Checkpoints
- Replication
```

# 20. One-Line Summary

**Atomicity and durability are implemented using recovery systems such as shadow copying and, more commonly, log-based recovery with undo/redo logs, checkpoints, and persistent storage.**